

LAPORAN AKHIR MAGANG MANDIRI

Rancang Bangun Sistem Portal Berita dan Content Management System (CMS) Redaksi Di PT. Ketik Media Siber

No. PKS: 308/UN63.7/IA-IF/2025

Diajukan untuk memenuhi persyaratan kelulusan
Program Magang Mandiri

oleh :

Farrel Aqeel Danendra / 23081010204

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN” JAWA TIMUR

2026

Lembar Pengesahan

Rancang Bangun Sistem Portal Berita dan Content Management System (CMS) Redaksi

oleh :

Farrel Aqeel Danendra / 23081010204

disetujui sebagai
Laporan Magang Mandiri

Surabaya, Juni 2026
Dosen Pembimbing

Yoga Ari Tofan, S.Kom., M.Kom
NIP. 199302032025061004

Abstraksi

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan Program Pembelajaran di Luar Kampus (PLK) yang dilaksanakan di PT. Ketik Media Siber selama periode 9 Februari 2026 hingga 19 Juni 2026. Selama kegiatan magang, penulis terlibat dalam proyek Rancang Bangun Sistem Portal Berita dan Content Management System (CMS) Redaksi yang bertujuan membangun platform pengelolaan dan publikasi berita berbasis web secara modern dan terstruktur.

Lingkup pekerjaan yang dilaksanakan meliputi perancangan arsitektur sistem menggunakan pendekatan decoupled (Laravel 12 sebagai backend dan Next.js 15 sebagai frontend), perancangan database komprehensif (CDM, PDM, ERD) terdiri dari 15 tabel, serta implementasi fitur-fitur utama seperti cronjob scheduled publish untuk penjadwalan otomatis publikasi artikel dan manajemen user (CRUD) untuk pengelolaan akun pengguna oleh admin. Penulis juga menyusun dokumentasi perancangan sistem menggunakan diagram UML meliputi Sequence Diagram, Activity Diagram, dan Class Diagram.

Hasil dari kegiatan magang ini adalah sebuah sistem portal berita dan CMS redaksi yang berfungsi penuh, dilengkapi dengan mekanisme penjadwalan publikasi otomatis dan fitur pengelolaan pengguna yang aman. Sistem telah diuji secara internal dan berjalan sesuai spesifikasi yang ditetapkan. Selain itu, penulis juga menyusun rekognisi mata kuliah yang mengaitkan pengalaman praktis selama magang dengan mata kuliah Analisis Kebutuhan, Pemrograman API, dan Uji Coba dan Implementasi.

Kata kunci: content management system, portal berita, Laravel, Next.js, scheduled publish, manajemen user.

Kata Pengantar

Puji syukur kehadiran Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Akhir Program Pembelajaran Luar Kampus (PLK) ini dengan baik. Laporan ini disusun sebagai salah satu syarat penyelesaian Program Pembelajaran Luar Kampus (PLK) pada Program Studi Informatika, Fakultas Ilmu Komputer, Universitas Pembangunan Nasional “Veteran” Jawa Timur. Program magang dilaksanakan di PT. Ketik Media Siber sebagai Software Development Intern, di mana penulis terlibat dalam pengembangan Sistem Portal Berita dan Content Management System (CMS) berbasis web.

Dalam penyusunan laporan ini, penulis memperoleh bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis menyampaikan ucapan terima kasih kepada:

1. Ibu Prof. Dr. Ir. Novirina Hendrasarie, M.T. selaku Dekan Fakultas Ilmu Komputer Universitas Pembangunan Nasional “Veteran” Jawa Timur.
2. Ibu Dr. Intan Yuniar Purbasari, S.Kom., M.Sc. selaku Koordinator Program Studi Informatika.
3. Bapak Eka Prakarsa Mandyartha, S.T., M.Kom. beserta tim dosen selaku penanggung jawab Program PLK.
4. Bapak Yoga Ari Tofan, S.Kom., M.Kom. selaku dosen pembimbing yang telah memberikan arahan dan masukan.
5. PT. Ketik Media Siber yang telah memberikan kesempatan magang dan terlibat dalam pengembangan proyek.
6. Bapak Kiagus Firdaus, A.Md., S.Kom., M.AP. selaku Founder PT. Ketik Media Siber.
7. Bapak Galih selaku Direktur IT PT. Ketik Media Siber.
8. Bapak Ali selaku mentor tim magang IT PT. Ketik Media Siber.
9. Seluruh staf dan karyawan PT. Ketik Media Siber, khususnya Divisi IT.
10. Rekan-rekan peserta magang serta orang tua dan keluarga yang selalu memberikan dukungan.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan. Semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca.

Surabaya, Juni 2026

Penulis

Farrel Aqeel Danendra

NPM. 23081010204

Daftar Isi

Lembar Pengesahan	i
Abstraksi	ii
Kata Pengantar	iii
Daftar Isi	iv
Bab I Pendahuluan	1
I.1 Latar Belakang	1
I.2 Lingkup	2
I.3 Tujuan	3
Bab II Organisasi Mitra PLK	4
II.1 Struktur Organisasi	4
II.2 Lingkup Pekerjaan	5
II.3 Deskripsi Pekerjaan	5
II.4 Jadwal Kerja	6
Bab III Rancang Bangun Sistem Portal Berita dan CMS Redaksi	7
III.1 Deskripsi Persoalan	7
III.2 Proses Pelaksanaan	8
III.3 Pencapaian Hasil	48
III.4 Hasil Testing	50
III.5 Rekognisi Mata Kuliah	58
Bab IV Penutup	61
IV.1 Kesimpulan	61
IV.2 Saran	62
Referensi	63
Lampiran A Log Activity	64

BAB I

PENDAHULUAN

I.1 Latar Belakang

Perkembangan teknologi digital telah mengubah cara masyarakat memperoleh dan mengakses informasi. Jika sebelumnya informasi banyak disebarkan melalui media cetak, kini berbagai berita dan informasi dapat diakses dengan cepat melalui platform berbasis web. Kondisi tersebut mendorong perusahaan media untuk memanfaatkan teknologi yang mampu mendukung pengelolaan serta publikasi konten secara lebih efektif dan efisien.

Program Pembelajaran di Luar Kampus (PLK) merupakan kegiatan yang memberikan kesempatan kepada mahasiswa untuk memperoleh pengalaman kerja secara langsung di dunia industri. Melalui program ini, mahasiswa dapat menerapkan ilmu yang telah dipelajari selama perkuliahan sekaligus memahami proses kerja profesional, baik dari sisi teknis maupun kerja sama dalam tim.

Dalam pelaksanaan PLK, penulis melaksanakan kegiatan magang di PT. Ketik Media Siber, sebuah perusahaan yang bergerak di bidang media digital. Selama kegiatan magang, penulis terlibat dalam pengembangan Sistem Portal Berita dan Content Management System (CMS) Redaksi yang digunakan untuk membantu proses pengelolaan dan publikasi berita pada platform digital.

Pada proyek tersebut, penulis berperan sebagai Software Development dengan tugas utama mengembangkan antarmuka halaman editor menggunakan React.js, Next.js, dan Tailwind CSS. Selain itu, penulis juga membantu proses perancangan desain antarmuka, integrasi API, serta pengujian fitur yang dikembangkan agar sistem dapat berjalan sesuai kebutuhan pengguna.

I.2 Lingkup

Lingkup kegiatan Program Pembelajaran di Luar Kampus (PLK) yang dilaksanakan di PT. Ketik Media Siber berfokus pada pengembangan Sistem Portal Berita dan Content Management System (CMS) Redaksi. Selama kegiatan magang berlangsung, penulis terlibat dalam beberapa kegiatan yang berkaitan dengan pengembangan sistem, yaitu sebagai berikut:

1. **Perancangan Arsitektur Sistem dan Basis Data** — Menyusun dokumentasi perancangan sistem menggunakan diagram UML (Use Case Diagram, Activity Diagram, Sequence Diagram, dan Class Diagram) sebagai acuan pengembangan perangkat lunak. Merancang struktur basis data menggunakan model Entity Relationship Diagram (ERD), Conceptual Data Model (CDM), dan Physical Data Model (PDM) untuk memastikan integritas dan efisiensi penyimpanan data.

2. **Setup dan Konfigurasi Proyek** — Menyiapkan struktur direktori dasar proyek, pemilihan tech stack, serta konfigurasi lingkungan pengembangan (development environment) untuk menjaga konsistensi dan skalabilitas kode. Mengelola repositori proyek serta menerapkan workflow kolaborasi tim menggunakan version control system (Git/GitHub).
3. **Pengembangan Backend dan API** — Membangun Application Programming Interface (API) sebagai jembatan komunikasi data yang efisien antara server dan client. Mengelola logika bisnis pada sisi server serta melakukan validasi data untuk memastikan keamanan dan reliabilitas pertukaran informasi antar modul sistem.

I.3 Tujuan

Program Pembelajaran di Luar Kampus (PLK) yang dilaksanakan di PT. Ketik Media Siber bertujuan untuk memberikan kesempatan kepada mahasiswa dalam mengenal lingkungan kerja secara langsung sekaligus mengaplikasikan pengetahuan yang telah diperoleh selama masa perkuliahan. Adapun tujuan pelaksanaan kegiatan PLK ini adalah sebagai berikut:

1. Menambah pemahaman mengenai pengembangan aplikasi web yang digunakan pada lingkungan kerja profesional.
2. Meningkatkan keterampilan dalam membangun dan mengembangkan antarmuka pengguna yang responsif dan mudah digunakan.
3. Memperdalam pemahaman mengenai penggunaan teknologi modern dalam pengembangan aplikasi berbasis web.
4. Memahami proses kolaborasi antar tim dalam pengembangan dan pemeliharaan sistem informasi.
5. Melatih kemampuan dalam menyelesaikan permasalahan teknis yang muncul selama proses pengembangan aplikasi.
6. Mengetahui alur pengelolaan konten dan publikasi berita melalui sistem yang digunakan oleh perusahaan media digital.
7. Mengembangkan kemampuan komunikasi, tanggung jawab, serta profesionalisme dalam lingkungan kerja.
8. Memperoleh pengalaman dan wawasan yang dapat mendukung persiapan karir di bidang teknologi informasi.

BAB II

ORGANISASI ATAU LINGKUNGAN ORGANISASI MITRA PLK

II.1 Struktur Organisasi

PT. Ketik Media Siber memiliki struktur organisasi yang terdiri atas beberapa divisi yang saling mendukung dalam menjalankan kegiatan operasional perusahaan. Setiap divisi memiliki peran dan tanggung jawab masing-masing, mulai dari pengelolaan media digital, pengembangan bisnis, administrasi perusahaan, hingga pengembangan teknologi informasi.

Dalam pelaksanaan Program Pembelajaran di Luar Kampus (PLK), penulis ditempatkan pada Divisi Teknologi yang berada di bawah koordinasi Chief Technology Officer (CTO). Divisi ini bertanggung jawab dalam pengembangan, pemeliharaan, dan pengelolaan sistem digital yang digunakan untuk mendukung proses bisnis perusahaan.

Selama kegiatan magang, penulis berperan sebagai Software Development pada proyek Sistem Portal Berita dan Content Management System (CMS) Redaksi. Penulis terlibat dalam berbagai aktivitas pengembangan sistem, seperti analisis kebutuhan, perancangan antarmuka, implementasi fitur, integrasi API, serta pengujian aplikasi.

II.2 Lingkup Pekerjaan

Divisi Teknologi di PT. Ketik Media Siber bertanggung jawab atas pengembangan dan pemeliharaan seluruh produk digital perusahaan. Lingkup pekerjaan divisi ini mencakup perancangan arsitektur sistem, pengembangan frontend dan backend, pengelolaan database dan infrastruktur server, serta pemeliharaan dan peningkatan kualitas produk digital yang telah diluncurkan. Dalam konteks proyek Sistem Portal Berita dan CMS Redaksi, penulis bekerja sama dengan mentor dan tim IT untuk mengembangkan fitur-fitur yang dibutuhkan oleh redaksi.

II.3 Deskripsi Pekerjaan

Selama pelaksanaan Program Magang Mandiri di PT. Ketik Media Siber, penulis terlibat secara langsung dalam pengembangan Sistem Portal Berita dan Content Management System (CMS) Redaksi. Kegiatan yang dilaksanakan mencakup keseluruhan siklus pengembangan perangkat lunak, mulai dari tahap perencanaan hingga pengujian. Berikut adalah tahapan pekerjaan yang dilakukan:

1. Perencanaan dan Analisis Kebutuhan

Pada tahap ini, penulis melakukan identifikasi dan analisis kebutuhan sistem yang akan dibangun. Penulis mempelajari alur kerja redaksi PT. Ketik Media Siber secara menyeluruh, mulai dari proses penulisan berita oleh jurnalis, proses peninjauan dan persetujuan oleh editor,

hingga proses publikasi artikel. Berdasarkan analisis tersebut, penulis mengidentifikasi dua fitur utama yang menjadi tanggung jawab pengembangan, yaitu fitur **penjadwalan publikasi artikel otomatis (scheduled publish)** yang memungkinkan artikel dipublikasikan secara otomatis pada waktu yang telah ditentukan, serta fitur **manajemen pengguna (user management)** yang memungkinkan admin mengelola akun pengguna meliputi pembuatan, pengeditan, penonaktifan, dan penghapusan akun.

2. Perencanaan Struktur Aplikasi

Pada tahap perencanaan struktur, penulis merancang arsitektur sistem menggunakan pendekatan *decoupled architecture* yang memisahkan backend dan frontend menjadi dua aplikasi terpisah. Backend dibangun menggunakan framework Laravel 12 dengan bahasa pemrograman PHP 8.4 dan database SQLite, sedangkan frontend dibangun menggunakan framework Next.js 15 dengan TypeScript dan Tailwind CSS. Penulis juga menerapkan pola *Service-Repository Pattern* untuk memisahkan logika bisnis dari operasi database. Selain itu, penulis menyusun perancangan database yang meliputi perancangan *Conceptual Data Model* (CDM), *Physical Data Model* (PDM), dan *Entity Relationship Diagram* (ERD) yang menghasilkan 15 tabel utama. Dokumentasi sistem juga disusun dalam bentuk diagram UML yang mencakup *Sequence Diagram*, *Activity Diagram*, dan *Class Diagram* untuk masing-masing fitur yang dikembangkan.

3. Implementasi

Pada tahap implementasi, penulis melakukan pengembangan kedua fitur utama yang telah direncanakan. Untuk fitur **scheduled publish**, penulis membangun Artisan Command sebagai *entry point* yang dipicu oleh Laravel Task Scheduler setiap menit, serta *ScheduledPublishService* yang berisi logika bisnis untuk memproses artikel terjadwal. Proses ini mencakup pembaruan status artikel menjadi **published**, pembaruan flag pada tabel **scheduled_articles**, serta pembaruan indeks pencarian (*search index*) agar artikel yang baru dipublikasikan dapat langsung ditemukan. Seluruh operasi dilakukan dalam satu *database transaction* untuk menjamin konsistensi data.

Untuk fitur **manajemen user**, penulis membangun operasi CRUD (*Create, Read, Update, Delete*) yang hanya dapat diakses oleh pengguna dengan role admin. Fitur ini mencakup pembuatan akun baru dengan *auto-generate password* dan pengiriman kredensial melalui email, pengeditan data pengguna dengan mekanisme *revoke token* otomatis apabila terjadi perubahan role, penonaktifan akun, serta penghapusan akun yang disertai pengarsipan seluruh artikel milik pengguna tersebut. Seluruh endpoint API dilindungi dengan middleware autentikasi JWT dan validasi role.

4. Uji Coba

Pada tahap uji coba, penulis melakukan pengujian terhadap seluruh fitur yang telah diimplementasikan untuk memastikan sistem berjalan sesuai spesifikasi yang telah ditetapkan.

Pengujian fitur **scheduled publish** mencakup pengujian penjadwalan artikel, verifikasi perubahan status artikel secara otomatis oleh cron job, serta verifikasi pembaruan indeks pencarian. Pengujian fitur **manajemen user** mencakup pengujian pembuatan user baru, validasi duplikasi email, pengujian pengiriman email kredensial, pengujian pengeditan data termasuk perubahan role dan penonaktifan akun, pengujian penghapusan user beserta penanganan dampaknya, serta pengujian kontrol akses untuk memastikan hanya admin yang dapat mengakses fitur ini. Hasil pengujian menunjukkan bahwa seluruh fitur berfungsi dengan baik dan sesuai dengan kebutuhan yang telah dianalisis pada tahap perencanaan.

II.4 Jadwal Kerja

Program Pembelajaran di Luar Kampus (PLK) dilaksanakan di PT. Ketik Media Siber selama kurang lebih empat bulan sepuluh hari, dimulai pada 9 Februari 2026 dan berakhir pada 19 Juni 2026. Kegiatan magang dilaksanakan enam hari dalam satu minggu, yaitu Senin hingga Sabtu. Jam kerja pada hari Senin sampai Jumat berlangsung pukul 09.00–17.00 WIB, sedangkan pada hari Sabtu pukul 09.00–13.00 WIB.

Minggu	Periode	Kegiatan	Jam
Minggu 1-3	9 Feb – 28 Feb 2026	Pengenalan perusahaan, pelatihan jurnalistik, penulisan dan publikasi berita, serta pemahaman alur kerja redaksi.	132
Minggu 4-8	2 Mar – 4 Apr 2026	Pengenalan proyek, Git dan GitHub, pengelolaan data website, serta pembelajaran workflow pengembangan sistem.	220
Minggu 9-10	6 Apr – 18 Apr 2026	Perancangan UI/UX dan dokumentasi perancangan sistem Website Klojen.	88
Minggu 11-14	20 Apr – 16 Mei 2026	Setup project dan pengembangan antarmuka Website Klojen berdasarkan desain yang telah dibuat.	176
Minggu 15-16	18 Mei – 30 Mei 2026	Integrasi API, pengembangan fitur pencarian dan filter, serta penyempurnaan tampilan website.	88
Minggu 17-18	1 Jun – 13 Jun 2026	Optimasi sistem, perbaikan bug, dan pengujian website.	88
Minggu 19	15 Jun – 19 Jun 2026	Finalisasi proyek, penyusunan dokumentasi, dan persiapan presentasi hasil magang.	36

Tabel 1. Jadwal Kerja Program Magang Mandiri

BAB III

RANCANG BANGUN SISTEM PORTAL BERITA DAN CMS REDAKSI

III.1 Deskripsi Persoalan

PT. Ketik Media Siber merupakan perusahaan yang bergerak di bidang media digital dan mengelola portal berita Klojen.com. Sebelum adanya sistem baru, proses pengelolaan konten berita dilakukan secara manual dengan alur kerja yang kurang terstruktur. Para jurnalis menulis berita, kemudian menyerahkan draf kepada editor melalui komunikasi informal, dan editor melakukan proses publikasi secara langsung tanpa sistem pelacakan revisi yang memadai. Kondisi ini menyebabkan beberapa permasalahan, antara lain: kesulitan dalam melacak riwayat perubahan artikel, tidak adanya mekanisme penjadwalan publikasi artikel, ketiadaan kontrol akses berbasis peran (role-based access control), serta tidak adanya sistem pencarian yang efisien.

Berdasarkan permasalahan tersebut, penulis ditugaskan untuk melakukan perancang bangun Sistem Portal Berita dan Content Management System (CMS) Redaksi. Sistem yang dikembangkan harus memenuhi kebutuhan berikut:

1. **Perancangan Arsitektur Sistem** — menyusun struktur proyek secara keseluruhan, meliputi pemilihan tech stack, konfigurasi repository, perancangan database (CDM, PDM, ERD), serta dokumentasi desain sistem menggunakan diagram UML.
2. **Fitur Cronjob Scheduled Publish** — membangun mekanisme penjadwalan publikasi artikel secara otomatis menggunakan cron job, di mana artikel yang telah diatur waktu tayangnya akan otomatis dipublikasikan ketika jadwal tiba.
3. **Fitur Manajemen User (CRUD)** — membangun fitur pengelolaan pengguna sistem oleh admin, meliputi pembuatan akun baru, pengeditan data, aktivasi/nonaktifkan akun, serta penghapusan pengguna dengan penanganan dampak terhadap data terkait.

III.2 Proses Pelaksanaan

III.2.1 Penyusunan Struktur Proyek dan Perancangan Database

Tahap pertama dalam pelaksanaan proyek adalah menyusun struktur proyek dari awal (from scratch) serta melakukan perancangan database yang komprehensif. Penulis bertanggung jawab dalam menentukan arsitektur sistem, memilih teknologi yang digunakan, serta merancang seluruh struktur database yang menjadi fondasi sistem.

Arsitektur sistem dirancang menggunakan pendekatan **decoupled architecture**, di mana backend dan frontend dipisah menjadi dua aplikasi terpisah yang berkomunikasi melalui REST API. Backend dibangun menggunakan **Laravel 12** dengan bahasa pemrograman PHP 8.4 dan

database SQLite. Frontend dibangun menggunakan **Next.js 15** dengan TypeScript dan framework CSS Tailwind CSS.

Pada sisi backend, penulis menerapkan pola **Service-Repository Pattern** yang memisahkan logika bisnis (Service) dari operasi database (Repository). Setiap entitas utama memiliki Service dan Repository tersendiri, seperti ArticleService, UserService, CategoryService, dan lain-lain. Seluruh operasi database dilakukan melalui Query Builder Laravel, bukan ORM Eloquent, untuk memberikan kontrol penuh terhadap query SQL yang dieksekusi.

Perancangan database dimulai dari **Conceptual Data Model (CDM)**, kemudian diturunkan menjadi **Physical Data Model (PDM)**, dan diimplementasikan dalam bentuk file migrasi Laravel. Database sistem terdiri dari 15 tabel utama:

No	Nama Tabel	Deskripsi
1	users	Data pengguna sistem (admin, editor, journalist, reader)
2	categories	Kategori berita (Politik, Ekonomi, Olahraga, dsb.)
3	tags	Tag artikel untuk pengelompokan topik
4	articles	Artikel berita dengan metadata lengkap
5	article_tags	Relasi many-to-many antara artikel dan tag
6	article_revisions	Riwayat revisi artikel (snapshot before update)
7	scheduled_articles	Jadwal publikasi artikel terjadwal
8	search_indexes	Index pencarian full-text
9	comments	Komentar pembaca pada artikel
10	media	File media (gambar) yang diunggah
11	bookmarks	Artikel yang disimpan oleh pembaca
12	refresh_tokens	Token refresh untuk autentikasi JWT
13	password_reset_tokens	Token reset password
14	page_views	Pencatatan statistik kunjungan halaman
15	comment_rate_limits	Pembatasan rate komentar per IP

Seluruh tabel menggunakan UUID sebagai primary key bertipe string untuk keamanan dan menghindari prediksi ID. Relasi antar tabel dirancang dengan foreign key constraint yang menjaga integritas data.

III.2.2 Implementasi Cronjob Scheduled Publish

Fitur *Scheduled Publish* memungkinkan editor atau admin untuk mengatur waktu publikasi artikel di masa depan. Ketika waktu yang dijadwalkan tiba, sistem secara otomatis mengubah status artikel dari *scheduled* menjadi *published* tanpa intervensi manual. Implementasi fitur ini melibatkan tiga komponen utama: (1) **Artisan Command** sebagai entry point yang dipanggil oleh scheduler, (2) **ScheduledPublishService** yang berisi seluruh logika bisnis penjadwalan, dan (3) **Laravel Task Scheduler** yang dikonfigurasi untuk menjalankan command setiap menit.

Berikut adalah potongan kode dari Artisan Command yang menjadi trigger utama proses *scheduled publish*:

```

PHP

class PublishScheduledArticles extends Command
{
    protected $signature = 'articles:publish-scheduled';
    protected $description = 'Publish semua artikel terjadwal yang sudah waktunya tayang';

    public function __construct(private readonly ScheduledPublishService $service)
    {
        parent::__construct();
    }

    public function handle(): int
    {
        $this->info('[ScheduledPublish] Memulai proses auto-publish...');
        $count = $this->service->run();
        if ($count === 0) {
            $this->line(' -> Tidak ada artikel yang perlu dipublish. ');
        } else {
            $this->info(" -> {$count} artikel berhasil dipublish dan diindeks.");
        }
        return Command::SUCCESS;
    }
}

```

Potongan kode 1. Artisan Command `articles:publish-scheduled` yang menjadi entry point cron job.

Pada potongan kode di atas, class **PublishScheduledArticles** adalah Artisan Command yang dibuat untuk menangani proses auto-publish artikel terjadwal. Command ini punya signature `articles:publish-scheduled` dan di dalamnya sudah di-inject **ScheduledPublishService** sebagai dependency. Jadi ketika command ini dijalankan, method `handle()` akan memanggil service untuk memproses artikel yang sudah waktunya tayang, lalu menampilkan output di console berapa artikel yang berhasil dipublish.

Inti dari proses penjadwalan terdapat pada method `run()` di **ScheduledPublishService**. Berikut adalah potongan kode yang melakukan query untuk mengambil artikel yang sudah waktunya tayang:

PHP

```
// Query artikel terjadwal yang sudah waktunya tayang
$due = DB::table('scheduled_articles')
    ->where('scheduled_at', '<=', now())
    ->where('is_published', false)
    ->get();
if ($due->isEmpty()) {
    return 0;
}
```

Potongan kode 2. Query pengambilan artikel terjadwal yang sudah jatuh tempo.

Kode di atas berfungsi untuk mengambil semua artikel yang sudah dijadwalkan dan waktunya sudah lewat. Query-nya mencari di tabel `scheduled_articles` dengan syarat `scheduled_at <= NOW()` (waktunya sudah lewat) dan `is_published = false` (belum pernah dipublish sebelumnya). Kalau hasilnya kosong, berarti tidak ada artikel yang perlu diproses dan method langsung return 0.

Setelah mendapatkan daftar artikel yang perlu dipublish, setiap artikel diproses dalam sebuah database transaction:

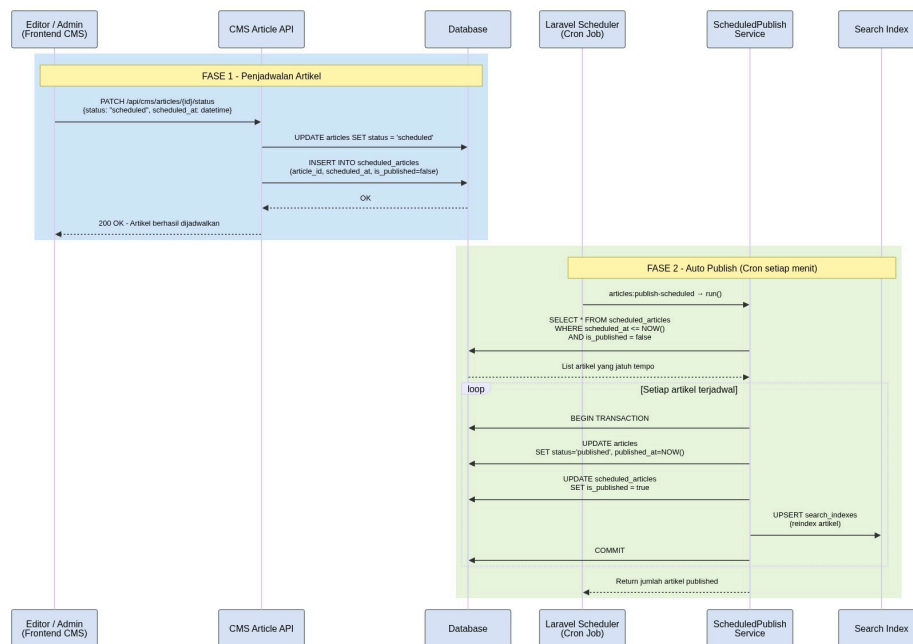
PHP

```
// Proses publish satu artikel dalam database transaction
DB::transaction(function () use ($scheduled, $articleId, $now) {
    DB::table('articles')
        ->where('id', $articleId)
        ->where('status', 'scheduled')
        ->update([
            'status'      => 'published',
            'published_at' => $now,
            'updated_at'  => $now,
        ]);
    DB::table('scheduled_articles')
        ->where('id', $scheduled->id)
        ->update(['is_published' => true]);
    $this->upsertSearchIndex($articleId, $now);
});
```

Potongan kode 3. Proses publish artikel dalam database transaction.

Potongan kode di atas menunjukkan proses publish satu artikel yang dibungkus dalam satu `DB::transaction()`. Tujuannya supaya ketiga operasi di dalamnya berjalan bersamaan — kalau salah satu gagal, semuanya di-rollback. Pertama, update status artikel jadi `published` dan isi `published_at` dengan waktu sekarang. Kedua, update flag `is_published` di tabel `scheduled_articles` jadi `true` supaya artikelnya tidak diproses lagi di cron berikutnya. Ketiga, reindex artikel ke tabel `search_indexes` biar artikel yang baru `published` langsung bisa ditemukan di fitur pencarian.

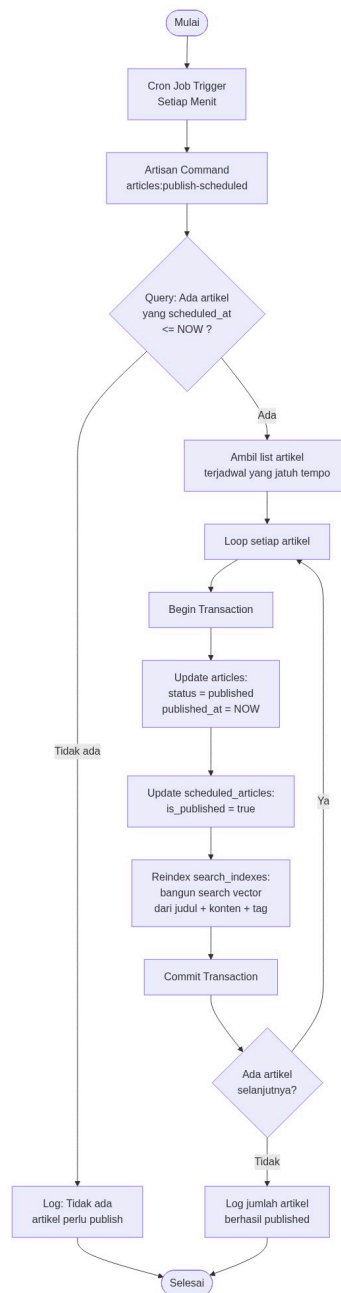
Berikut adalah Sequence Diagram yang menggambarkan alur kerja fitur Scheduled Publish:



Gambar 3.1 Sequence Diagram — Cronjob Scheduled Publish

Diagram di atas menjelaskan alur lengkap fitur Scheduled Publish dari awal sampai akhir. Di **Fase 1**, editor atau admin menjadwalkan artikel lewat API dengan menentukan kapan artikelnya harus tayang. Sistem simpan statusnya sebagai **scheduled** dan catat jadwalnya di tabel **scheduled_articles**. Di **Fase 2**, cron job jalan setiap menit dan ngecek apakah ada artikel yang sudah waktunya tayang. Kalau ada, sistem update statusnya jadi **published**, tandai jadwalnya sudah diproses, dan update search index supaya artikelnya langsung muncul di pencarian.

Berikut adalah Activity Diagram yang menggambarkan alur aktivitas fitur Scheduled Publish:



Gambar 3.2 Activity Diagram — Scheduled Publish

Activity Diagram di atas menggambarkan proses secara detail dimulai dari trigger cron job, pengecekan artikel yang jatuh tempo, loop pemrosesan setiap artikel, hingga logging hasil eksekusi.

III.2.3 Implementasi Manajemen User (CRUD) pada Role Admin

Fitur Manajemen User merupakan fitur yang hanya dapat diakses oleh pengguna dengan role **Admin**. Fitur ini mencakup operasi Create, Read, Update, dan Delete. Sistem manajemen user dirancang dengan memperhatikan aspek keamanan, seperti pencegahan admin menghapus akun sendiri, auto-generate password yang aman, dan pengiriman kredensial melalui email.

Arsitektur fitur ini mengikuti pola Service-Repository: **UserController** menangani HTTP request/response, **UserService** menangani logika bisnis, dan **UserRepository** menangani operasi database.

Berikut adalah potongan kode untuk operasi Create User:

PHP

```
// UserService::createUser() - Membuat user baru oleh Admin
public function createUser(array $data): User
{
    // 1. Cek duplikat email
    if ($this->userRepository->findByEmail($data['email'])) {
        throw new \RuntimeException('EMAIL_ALREADY_EXISTS', 409);
    }

    // 2. Generate password sementara: 12 karakter acak
    $plainPassword = $this->generatePassword();
    // 3. Buat user - password otomatis di-hash via model cast 'hashed'
    $user = $this->userRepository->create([
        'name' => $data['name'], 'email' => $data['email'],
        'password' => $plainPassword, 'role' => $data['role'],
        'is_active' => true,
    ]);
    // 4. Kirim email berisi kredensial ke user baru
    Mail::to($user->email)->send(new NewUserCredentials(
        name: $user->name, email: $user->email,
        plainPassword: $plainPassword, role: $user->role,
    ));
    return $user;
}
```

Potongan kode 4. Method createUser() pada UserService untuk membuat user baru.

Di method `createUser()` ini, pertama sistem cek dulu apakah email yang diinput sudah terdaftar atau belum. Kalau sudah, langsung throw error. Kalau belum, sistem generate password acak 12 karakter, terus insert data user ke database. Passwordnya otomatis di-hash oleh Laravel jadi tidak disimpan dalam bentuk plain text. Setelah user berhasil dibuat, sistem kirim email ke alamat user baru yang isinya nama, email, password sementara, dan role-nya.

Berikut adalah potongan kode untuk generate password acak:

PHP

```
// Generate password acak yang aman (12 karakter)
private function generatePassword(): string
{
    $upper = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
    $lower = 'abcdefghijklmnopqrstuvwxyz';
    $digits = '23456789';
    $symbols = '@#%&!';
    $password = $upper[random_int(0, strlen($upper) - 1)];
    $password .= $lower[random_int(0, strlen($lower) - 1)];
    $password .= $digits[random_int(0, strlen($digits) - 1)];
    $password .= $symbols[random_int(0, strlen($symbols) - 1)];
    $all = $upper . $lower . $digits . $symbols;
}
```

```

    for ($i = 0; $i < 8; $i++) {
        $password .= $all[random_int(0, strlen($all) - 1)];
    }
    return str_shuffle($password);
}

```

Potongan kode 5. Method generatePassword() untuk menghasilkan password acak yang aman.

Fungsi `generatePassword()` ini digunakan untuk membuat password sementara sepanjang 12 karakter. Passwordnya dijamin ada minimal satu huruf besar, satu huruf kecil, satu angka, dan satu simbol. Karakter-karakter yang gampang bikin bingung seperti huruf O, huruf l, angka 1, dan angka 0 sengaja tidak dipakai supaya user tidak salah ketik waktu input password pertama kali.

Operasi Update User memiliki logika khusus yang mencegah admin mengubah akun sendiri serta melakukan revoke token jika role user diubah:

```

PHP

// UserService::updateUser() - Mengupdate data user oleh Admin
public function updateUser(string $id, array $data, string $currentUserId): User
{
    if ($id === $currentUserId) {
        throw new \RuntimeException('CANNOT_MODIFY_SELF', 403);
    }
    $user = $this->userRepository->findById($id);
    if (!$user) throw new \RuntimeException('USER_NOT_FOUND', 404);
    if (isset($data['email']) &&
        $this->userRepository->findByEmailExcept($data['email'], $id)) {
        throw new \RuntimeException('EMAIL_ALREADY_EXISTS', 409);
    }
    $oldRole = $user->role;
    $updateData = array_filter($data, fn($v) => !is_null($v));
    $this->userRepository->update($user, $updateData);
    // Jika role diubah, revoke semua token (logout paksa)
    if (isset($updateData['role']) && $updateData['role'] !== $oldRole) {
        $this->refreshTokenRepository->revokeAllForUser($id);
    }
    return $user->refresh();
}

```

Potongan kode 6. Method updateUser() dengan validasi dan revoke token otomatis.

Di method `updateUser()` ini ada beberapa pengecekan yang dilakukan. Pertama, sistem cek apakah admin mencoba mengubah akunnya sendiri — kalau iya, langsung ditolak biar admin tidak sengaja mengunci akunnya. Kedua, kalau emailnya diganti, cek dulu apakah email baru itu sudah dipakai user lain. Ketiga, hanya field yang dikirim saja yang di-update, jadi tidak perlu kirim semua field. Yang menarik, kalau role user diubah, semua refresh token milik user itu langsung di-revoke. Efeknya user langsung logout dari semua perangkat yang aktif, jadi perubahan role langsung berlaku saat itu juga.

Operasi Delete User merupakan operasi yang paling kompleks karena melibatkan penanganan dampak terhadap data terkait:

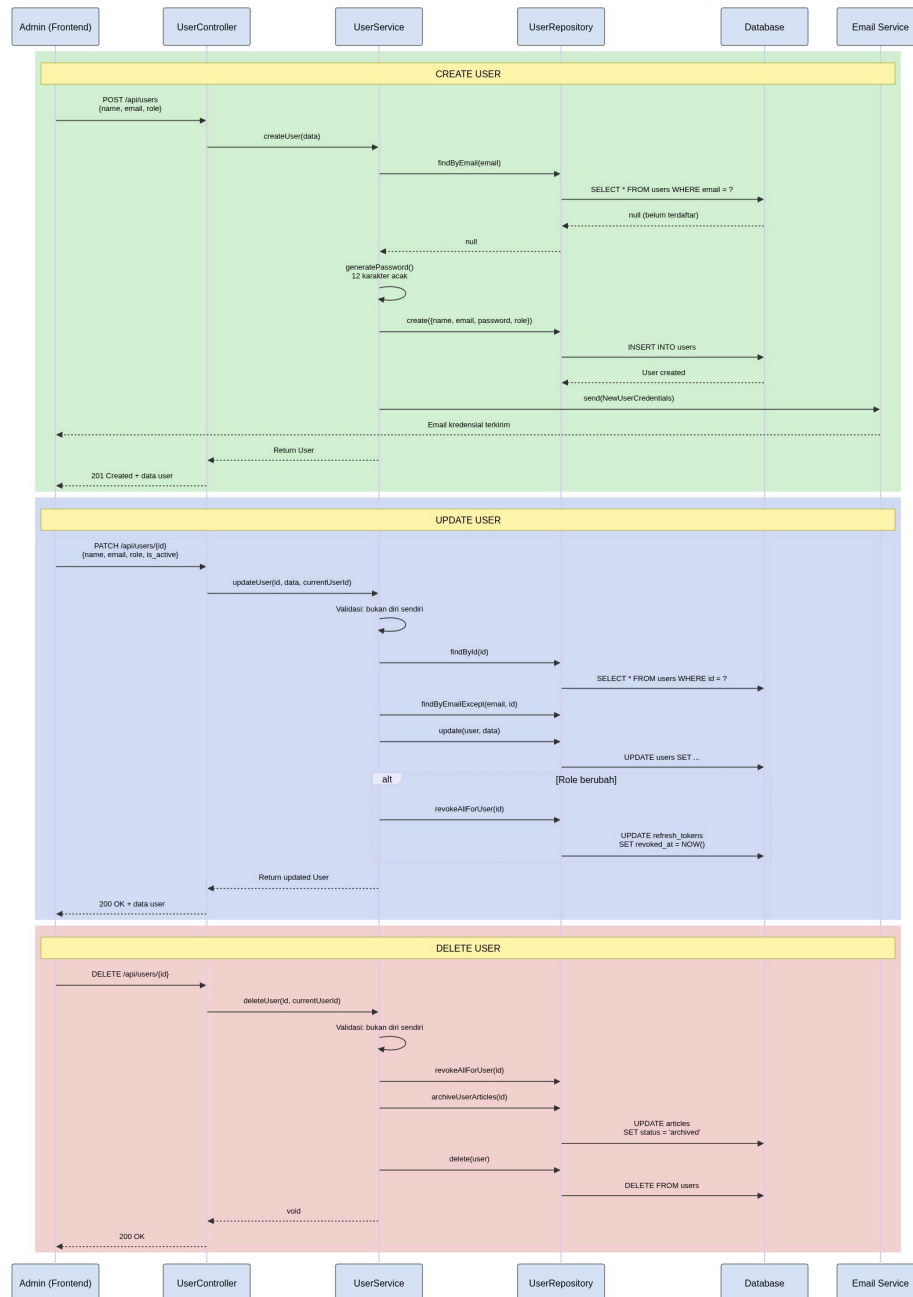
PHP

```
// UserService::deleteUser() - Menghapus user oleh Admin
public function deleteUser(string $id, string $currentUserId): void
{
    if ($id === $currentUserId) {
        throw new \RuntimeException('CANNOT_MODIFY_SELF', 403);
    }
    $user = $this->userRepository->findById($id);
    if (!$user) throw new \RuntimeException('USER_NOT_FOUND', 404);
    $this->refreshTokenRepository->revokeAllForUser($id);
    $this->articleRepository->archiveUserArticles($id);
    $this->userRepository->delete($user);
}
```

Potongan kode 7. Method deleteUser() dengan penanganan dampak terhadap data terkait.

Method `deleteUser()` ini melakukan tiga langkah secara berurutan. Pertama, revoke semua refresh token user supaya user langsung logout dari semua perangkat. Kedua, arsipkan semua artikel milik user tersebut dengan mengubah statusnya jadi **archived** — artikelnya tidak dihapus, cuma diarsipkan supaya sejarah kontennya tetap ada. Ketiga, baru hapus data user dari database.

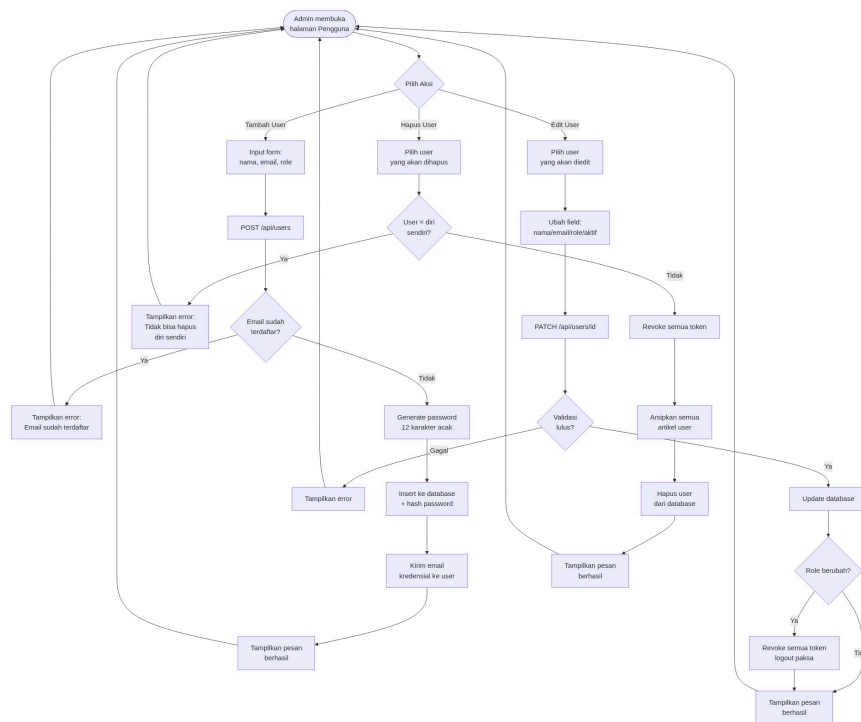
Berikut adalah Sequence Diagram yang menggambarkan alur kerja CRUD User:



Gambar 3.3 Sequence Diagram — Manajemen User (CRUD)

Diagram di atas menggambarkan tiga skenario operasi CRUD: Create User (blok hijau), Update User (blok biru), dan Delete User (blok merah).

Berikut adalah Activity Diagram yang menggambarkan alur aktivitas fitur Manajemen User:



Gambar 3.4 Activity Diagram — Manajemen User

Activity Diagram di atas menggambarkan keseluruhan alur operasi manajemen user dari perspektif admin, termasuk jalur validasi dan penanganan error pada setiap operasi.

Selain operasi CRUD, sistem juga dilengkapi dengan mekanisme **pengiriman email notifikasi** yang dikirim secara otomatis ketika admin membuat akun user baru. Email dikirim menggunakan class `NewUserCredentials` yang merupakan turunan dari `Mailable` milik Laravel. Berikut adalah potongan kode class `Mailable` yang digunakan:

PHP

```
// NewUserCredentials Mailable - Email kredensial untuk user baru
class NewUserCredentials extends Mailable
{
    use Queueable, SerializesModels;

    public function __construct(
        public readonly string $name,
        public readonly string $email,
        public readonly string $plainPassword,
        public readonly string $role,
    ) {}

    public function envelope(): Envelope
    {
        return new Envelope(
            subject: 'Selamat Datang di Portal Berita Klojen - Kredensial Akun Anda',
        );
    }

    public function content(): Content
```

```

    {
        return new Content(
            view: 'emails.new-user-credentials',
        );
    }
}

```

Potongan kode 9. Class NewUserCredentials Mailable untuk email kredensial user baru.

Class `NewUserCredentials` ini menerima empat parameter melalui constructor, yaitu `name`, `email`, `plainPassword` (password sementara dalam plain text), dan `role` user yang baru dibuat. Method `envelope()` menentukan subject email, sedangkan method `content()` menentukan view Blade yang digunakan sebagai template email, yaitu `emails.new-user-credentials`.

Template email yang digunakan berisi informasi lengkap yang dibutuhkan user baru untuk login pertama kali, meliputi nama pengguna, alamat email, password sementara, dan role yang diberikan. Email juga menyertakan peringatan agar user segera mengganti password setelah login pertama kali melalui menu Pengaturan Profil. Password sementara yang dikirim melalui email bersifat *one-time use* dan user diharapkan langsung mengubahnya demi keamanan akun. Pengiriman email dilakukan secara sinkron menggunakan Laravel Mail facade dengan driver SMTP yang dikonfigurasi melalui file `.env`.

III.2.4 Dokumentasi API Endpoint

Seluruh fitur yang dikembangkan diekspos melalui REST API yang dapat diakses oleh frontend. Berikut adalah dokumentasi API endpoint untuk fitur-fitur yang diimplementasikan oleh penulis.

A. Manajemen User (Admin Only)

Seluruh endpoint di bawah ini dilindungi oleh middleware `auth:api` dan `admin`, sehingga hanya dapat diakses oleh pengguna yang telah terautentikasi dengan role admin.

PHP

```

// routes/api.php - Definisi route untuk User Management
Route::middleware(['auth:api', 'admin'])->prefix('users')->group(function () {
    Route::get('/', [UserController::class, 'index']); // GET /api/users
    Route::post('/', [UserController::class, 'store']); // POST /api/users
    Route::get('/{id}', [UserController::class, 'show']); // GET /api/users/{id}
    Route::patch('/{id}', [UserController::class, 'update']); // PATCH /api/users/{id}
    Route::delete('/{id}', [UserController::class, 'destroy']); // DELETE /api/users/{id}
});

```

Potongan kode 8. Definisi route API untuk Manajemen User.

No	Method	Endpoint	Deskripsi
----	--------	----------	-----------

1	GET	/api/users	Mengambil daftar seluruh user dalam sistem
2	POST	/api/users	Membuat user baru (auto-generate password + kirim email)
3	GET	/api/users/{id}	Mengambil detail satu user berdasarkan ID
4	PATCH	/api/users/{id}	Mengupdate data user (nama, email, role, is_active)
5	DELETE	/api/users/{id}	Menghapus user (revoke token + arsipkan artikel)

Berikut adalah detail request dan response untuk masing-masing endpoint:

Endpoint	Request Body	Response Success	Response Error
POST /api/users	name (string), email (string), role (string)	201 Created + data user	409 EMAIL_ALREADY_EXISTS
PATCH /api/users/{id}	name?, email?, role?, is_active?	200 OK + data user	403 CANNOT_MODIFY_SELF, 404 USER_NOT_FOUND, 409 EMAIL_ALREADY_EXISTS
DELETE /api/users/{id}	-	200 OK + pesan berhasil	403 CANNOT_MODIFY_SELF, 404 USER_NOT_FOUND

B. Scheduled Publish (Cron Job)

Fitur Scheduled Publish tidak diekspos sebagai REST API, melainkan dijalankan secara internal melalui Artisan Command yang dipicu oleh Laravel Task Scheduler. Command dijalankan otomatis setiap menit oleh cron daemon di server.

Command	Signature	Jadwal	Deskripsi
PublishScheduledArticles	articles:publish-scheduled	Setiap menit (* * * * *)	Memproses seluruh artikel terjadwal yang sudah waktunya tayang

Command ini juga dapat dijalankan secara manual melalui terminal:

```
php artisan articles:publish-scheduled
```

III.2.5 Perancangan Conceptual Data Model, Physical Data Model, dan ERD

Perancangan database dilakukan secara bertahap dimulai dari **Conceptual Data Model (CDM)** yang menggambarkan entitas-entitas utama sistem beserta relasinya secara abstrak,

tanpa memperhatikan detail implementasi teknis. Pada tahap CDM, penulis mengidentifikasi delapan entitas utama: *User*, *Article*, *Category*, *Tag*, *Comment*, *Bookmark*, *ArticleRevision*, dan *ScheduledArticle*. Setiap entitas memiliki atribut-atribut kunci yang merepresentasikan informasi yang perlu disimpan.

Dari CDM, penulis menurunkan menjadi **Physical Data Model (PDM)** yang menambahkan detail teknis seperti tipe data, panjang field, constraint, dan index. Setiap tabel menggunakan UUID bertipe string (36 karakter) sebagai primary key untuk menghindari prediksi ID dan meningkatkan keamanan. Pemilihan UUID ini juga memudahkan proses migrasi data antar lingkungan (development, staging, production) karena tidak ada konflik auto-increment.

Tahap terakhir adalah **Entity Relationship Diagram (ERD)** yang menggambarkan relasi antar tabel secara lengkap beserta kardinalitasnya. Beberapa relasi penting yang dirancang meliputi: (1) relasi one-to-many antara *users* dan *articles* melalui *author_id*, (2) relasi many-to-many antara *articles* dan *tags* melalui tabel junction *article_tags*, (3) relasi one-to-many antara *articles* dan *comments* dengan self-referencing *parent_id* untuk mendukung nested comment, serta (4) relasi one-to-many antara *users* dan *refresh_tokens* untuk manajemen sesi autentikasi.

Foreign key constraint dirancang dengan strategi cascade yang sesuai untuk setiap relasi. Tabel *articles* memiliki constraint *restrictOnDelete* pada kolom *author_id* untuk mencegah penghapusan user yang memiliki artikel — sebagai gantinya, sistem menggunakan mekanisme arsip artikel sebelum penghapusan user. Sebaliknya, tabel *comments* dan *bookmarks* memiliki constraint *cascadeOnDelete* sehingga ketika artikel dihapus, seluruh komentar dan bookmark terkait otomatis ikut terhapus.

III.2.6 Implementasi Service-Repository Pattern secara Detail

Penulis menerapkan pola arsitektur **Service-Repository Pattern** secara konsisten di seluruh modul backend. Pola ini memisahkan tanggung jawab kode menjadi tiga layer yang jelas: **Controller** (menangani HTTP request/response dan validasi input), **Service** (menangani logika bisnis dan orkestrasi antar komponen), dan **Repository** (menangani operasi database).

Sebagai contoh implementasi, berikut adalah struktur class untuk modul *Scheduled Publish*:

- **PublishScheduledArticles** (Artisan Command) — berperan sebagai *entry point* yang dipanggil oleh scheduler. Command ini hanya bertanggung jawab memanggil service dan menampilkan output ke console.
- **ScheduledPublishService** — berisi seluruh logika bisnis penjadwalan, meliputi query artikel yang jatuh tempo, proses publish dalam transaction, dan reindex search index.

- **ScheduledArticleRepository** — menyediakan method untuk query tabel `scheduled_articles` seperti `findDueArticles()` dan `markAsPublished()`.

Pemisahan layer ini memberikan beberapa keuntungan signifikan. Pertama, setiap layer dapat diuji secara independen menggunakan *mocking* — service layer dapat diuji tanpa koneksi database nyata dengan cara me-mock repository. Kedua, penggantian implementasi repository (misalnya dari Query Builder ke stored procedure) tidak mempengaruhi service layer karena keduanya berkomunikasi melalui interface (`ScheduledArticleRepositoryInterface`). Ketiga, kode menjadi lebih mudah dibaca karena setiap class memiliki tanggung jawab tunggal (*Single Responsibility Principle*).

Pola yang sama juga diterapkan pada modul User Management, di mana `UserController` mendelegasikan seluruh operasi ke `UserService`, yang kemudian menggunakan `UserRepository` untuk operasi database. `UserService` juga berkoordinasi dengan `RefreshTokenRepository` untuk revoke token dan `ArticleRepository` untuk arsipkan artikel saat user dihapus.

III.2.7 Implementasi Database Migration dan Penggunaan UUID

Seluruh struktur database diimplementasikan menggunakan **Laravel Migration**, yang memungkinkan versioning skema database dan kolaborasi antar developer. Penulis membuat file migrasi untuk setiap tabel dengan penamaan yang mengikuti konvensi timestamp Laravel, memastikan urutan eksekusi migrasi yang benar.

Salah satu keputusan teknis penting yang diambil adalah penggunaan **UUID (Universally Unique Identifier)** sebagai primary key untuk seluruh tabel utama, kecuali tabel `page_views` yang menggunakan auto-increment integer karena volume data yang tinggi dan tidak memerlukan UUID. UUID di-generate menggunakan method `Str::uuid()` dari Laravel yang menghasilkan string 36 karakter dengan format standar (8-4-4-4-12).

Keuntungan penggunaan UUID meliputi: (1) **keamanan** — ID tidak dapat diprediksi sehingga mengurangi risiko enumeration attack, (2) **desentralisasi** — ID dapat di-generate di aplikasi tanpa perlu koneksi ke database, dan (3) **portabilitas** — data dari berbagai sumber dapat di-merge tanpa konflik ID. Namun, UUID juga memiliki trade-off: ukuran index yang lebih besar dibandingkan integer dan performa query yang sedikit lebih lambat pada dataset sangat besar.

Penulis juga membuat beberapa file migrasi tambahan untuk modifikasi skema setelah tabel awal dibuat, seperti penambahan kolom `published_by` dan `locked_by` pada tabel `articles`, serta perubahan constraint pada kolom `parent_id` di tabel `comments`. Pendekatan ini memastikan seluruh perubahan skema tercatat dalam version control dan dapat di-rollback jika diperlukan.

III.2.8 Konfigurasi Laravel Task Scheduler

Fitur **Scheduled Publish** menggunakan **Laravel Task Scheduler** yang merupakan abstraction layer di atas cron daemon. Penulis mengonfigurasi scheduler di file `routes/console.php` untuk menjalankan Artisan Command `articles:publish-scheduled` setiap menit menggunakan method `everyMinute()`.

Konfigurasi scheduler ini menggantikan kebutuhan untuk menulis cron expression secara manual di server. Dengan Laravel Scheduler, seluruh jadwal task terdokumentasi dalam kode dan dapat di-version control bersama kode aplikasi. Di server production, hanya diperlukan satu entry cron (`* * * * * php artisan schedule:run`) yang kemudian mendelegasikan penjadwalan detail ke Laravel.

Penulis juga menambahkan beberapa konfigurasi tambahan untuk meningkatkan reliabilitas scheduler: (1) method `withoutOverlapping()` untuk mencegah dua instance command berjalan secara bersamaan jika eksekusi sebelumnya belum selesai, (2) method `runInBackground()` untuk menjalankan command secara asynchronous sehingga tidak memblokir scheduler dari menjalankan task lain, serta (3) konfigurasi timezone agar jadwal eksekusi sesuai dengan zona waktu server.

Artisan Command `PublishScheduledArticles` dirancang dengan prinsip idempotensi — command dapat dijalankan berulang kali tanpa efek samping jika tidak ada artikel yang perlu diproses. Ketika tidak ada artikel jatuh tempo, command langsung mengembalikan nilai 0 tanpa melakukan operasi database apapun. Pendekatan ini memastikan scheduler yang berjalan setiap menit tidak memberikan beban unnecessary pada database.

III.2.9 Autentikasi JWT dan Role-Based Access Control

Sistem menggunakan **JSON Web Token (JWT)** sebagai mekanisme autentikasi utama. Penulis mengonfigurasi JWT menggunakan library `php-open-source-saver/jwt-auth` yang terintegrasi dengan Laravel Auth Guard. Setiap user yang berhasil login menerima dua token: **access token** (berlaku 60 menit) yang digunakan untuk mengakses API, dan **refresh token** (berlaku 14 hari) yang digunakan untuk mendapatkan access token baru ketika yang lama kedaluwarsa.

Refresh token disimpan di tabel `refresh_tokens` dengan hash SHA-256 untuk keamanan. Setiap record menyimpan informasi `user_id`, `token_hash`, `device_info`, `ip_address`, `is_revoked`, dan `expires_at`. Informasi device dan IP address membantu admin mengidentifikasi sesi yang mencurigakan.

Untuk kontrol akses, penulis mengimplementasikan **Role-Based Access Control (RBAC)** menggunakan middleware Laravel. Empat role didefinisikan: **admin** (akses penuh termasuk manajemen user), **editor** (tinjauan dan publikasi artikel), **journalist** (pembuatan artikel), dan **reader** (pembaca portal berita). Middleware `admin` (`EnsureAdmin`) memvalidasi bahwa user yang mengakses endpoint memiliki role admin sebelum request diproses. Endpoint

manajemen user (`/api/users/*`) dilindungi oleh middleware ini sehingga journalist, editor, maupun reader tidak dapat mengaksesnya.

Mekanisme **token revocation** diimplementasikan untuk menangani perubahan role dan penghapusan user. Ketika admin mengubah role seorang user, seluruh refresh token user tersebut di-revoke melalui method `revokeAllForUser()` di `RefreshTokenRepository`. Efeknya, user langsung logout dari semua perangkat aktif dan harus login ulang untuk mendapatkan token baru dengan role yang diperbarui.

III.2.10 Database Transaction dan ACID Properties

Penulis menerapkan **database transaction** pada operasi-operasi yang melibatkan perubahan di beberapa tabel secara bersamaan, khususnya pada fitur Scheduled Publish. Tujuannya adalah menjamin sifat **ACID** (*Atomicity, Consistency, Isolation, Durability*) dari operasi tersebut.

Atomicity dijamin dengan membungkus seluruh operasi publish dalam satu `DB::transaction()`. Jika salah satu operasi gagal (misalnya update status artikel berhasil tetapi reindex search index gagal), seluruh perubahan akan di-rollback sehingga data tetap konsisten. Tanpa transaction, kegagalan parsial dapat menyebabkan artikel berstatus `published` tetapi tidak ter-index di pencarian, atau sebaliknya.

Consistency dijaga melalui kombinasi transaction dan foreign key constraint. Setiap operasi publish mencakup tiga langkah yang harus berhasil semua: (1) update status dan `published_at` di tabel `articles`, (2) update flag `is_published` di tabel `scheduled_articles`, dan (3) upsert record di tabel `search_indexes`. Ketiga langkah ini merupakan satu kesatuan logis yang tidak boleh terpecah.

Isolation dicapai melalui mekanisme lock implisit yang disediakan oleh database engine SQLite. Ketika satu instance cron job sedang memproses artikel, instance lain yang berjalan secara bersamaan tidak akan dapat memodifikasi record yang sama sampai transaction pertama selesai. Hal ini dipadukan dengan konfigurasi `withoutOverlapping()` pada scheduler untuk memastikan tidak ada dua instance command yang berjalan bersamaan.

Durability dijamin oleh database engine itu sendiri — setelah transaction berhasil di-commit, perubahan data akan tersimpan secara permanen di disk dan tidak akan hilang meskipun server restart.

III.2.11 Template Email dan Blade View

Fitur pengiriman email kredensial menggunakan class **NewUserCredentials** yang merupakan turunan dari **Mailable** milik Laravel. Class ini dirancang dengan pendekatan constructor injection, di mana data user (nama, email, password sementara, dan role) diteruskan melalui constructor dan disimpan sebagai property public yang dapat diakses di view.

Template email dibuat menggunakan **Blade template engine** milik Laravel di file `resources/views/emails/new-user-credentials.blade.php`. Template ini dirancang dengan HTML yang responsif dan informatif, mencakup: (1) header dengan logo perusahaan, (2) pesan sambutan yang menyebutkan nama user, (3) tabel berisi informasi akun (email, password sementara, dan role), (4) tombol link ke halaman login, serta (5) peringatan keamanan agar user segera mengganti password setelah login pertama kali.

Pengiriman email dilakukan secara sinkron menggunakan `Mail::to($user->email)->send(new NewUserCredentials(...))`. Meskipun pendekatan sinkron lebih sederhana, penulis menyadari bahwa untuk skala produksi, pendekatan asynchronous menggunakan Laravel Queue akan lebih tepat agar proses pembuatan user tidak terblokir oleh waktu pengiriman email. Namun, untuk kebutuhan proyek saat ini dengan volume pembuatan user yang rendah, pendekatan sinkron sudah memadai.

Penulis juga mengonfigurasi driver email melalui file `.env` menggunakan SMTP. Konfigurasi ini memungkinkan pengujian pengiriman email di lingkungan development menggunakan service seperti Mailtrap, sementara di production menggunakan SMTP server yang sesungguhnya.

III.2.12 Konfigurasi Proyek dan Dependency Management

Penulis bertanggung jawab dalam menyiapkan dan mengonfigurasi seluruh proyek dari awal. Proses ini mencakup beberapa langkah penting yang menjadi fondasi bagi pengembangan fitur-fitur selanjutnya.

Pemilihan Tech Stack — Penulis melakukan analisis terhadap beberapa alternatif teknologi sebelum menetapkan pilihan akhir. Untuk backend, Laravel 12 dipilih karena ekosistemnya yang matang, dokumentasi yang lengkap, dan dukungan built-in untuk fitur yang dibutuhkan (Task Scheduler, Mailable, Queue, Migration). Untuk frontend, Next.js 15 dipilih karena kemampuan SSR/ISR yang penting untuk SEO portal berita. Tailwind CSS dipilih untuk styling karena utility-first approach yang mempercepat pengembangan UI.

Struktur Direktori — Penulis menyusun struktur direktori backend mengikuti konvensi Laravel dengan penambahan folder `Repositories` dan `Contracts` untuk mendukung Service-Repository Pattern. Setiap entitas memiliki tiga file: `Controller`, `Service`, dan `Repository`, plus satu file `Interface` di folder `Contracts`. Struktur ini memudahkan navigasi kode karena lokasi setiap class dapat diprediksi berdasarkan nama entitasnya.

Konfigurasi Environment — File `.env` dikonfigurasi untuk setiap lingkungan (development dan production) dengan nilai yang berbeda untuk database connection, JWT secret, SMTP credentials, dan CORS origins. Penulis juga menyiapkan file `.env.example` yang berisi template konfigurasi tanpa nilai sensitif, sehingga developer lain dapat dengan mudah menyiapkan lingkungan development mereka sendiri.

Dependency Management — Seluruh dependency PHP dikelola menggunakan **Composer** dengan file `composer.json` dan `composer.lock` yang di-commit ke repository. Pendekatan ini memastikan setiap developer dan server production menggunakan versi dependency yang sama persis. Penulis menambahkan beberapa package penting seperti `php-open-source-saver/jwt-auth` untuk JWT authentication dan `intervention/image` untuk pemrosesan gambar.

III.2.13 Detail Skema Database

Pada bagian ini dijelaskan secara rinci struktur kolom untuk setiap tabel dalam database sistem. Penjelasan ini mencakup nama kolom, tipe data, constraint, dan deskripsi fungsinya dalam sistem.

A. Tabel users

Tabel `users` menyimpan data seluruh pengguna sistem. Tabel ini menggunakan auto-increment integer sebagai primary key karena volume user yang relatif kecil dan kebutuhan query yang cepat.

Kolom	Tipe Data	Constraint	Deskripsi
id	BIGINT	PK, Auto Increment	Identifier unik pengguna
name	VARCHAR(255)	NOT NULL	Nama lengkap pengguna
email	VARCHAR(255)	UNIQUE, NOT NULL	Alamat email untuk login
role	ENUM	DEFAULT 'reader'	Peran: admin, editor, journalist, reader
is_active	BOOLEAN	DEFAULT true	Status akun aktif/nonaktif
avatar_url	VARCHAR(255)	NULLABLE	URL foto profil pengguna
bio	TEXT	NULLABLE	Biografi singkat pengguna
password	VARCHAR(255)	NOT NULL	Password ter-hash (bcrypt)
email_verified_at	TIMESTAMP	NULLABLE	Waktu verifikasi email
created_at	TIMESTAMP	AUTO	Waktu pembuatan akun
updated_at	TIMESTAMP	AUTO	Waktu update terakhir

B. Tabel articles

Tabel **articles** menyimpan seluruh artikel berita dengan metadata lengkap. Menggunakan UUID sebagai primary key.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik artikel
author_id	BIGINT	FK → users, RESTRICT ON DELETE	ID penulis artikel
published_by	BIGINT	FK → users, NULL ON DELETE, NULLABLE	ID editor yang mempublikasi
locked_by	BIGINT	FK → users, NULL ON DELETE, NULLABLE	ID editor yang mengunci artikel
category_id	UUID	FK → categories, RESTRICT, NULLABLE	ID kategori artikel
title	VARCHAR(255)	NOT NULL	Judul artikel
slug	VARCHAR(255)	UNIQUE, NOT NULL	URL-friendly identifier
excerpt	TEXT	NULLABLE	Ringkasan singkat artikel
content	LONGTEXT	NOT NULL	Konten lengkap artikel (HTML)
featured_image_url	VARCHAR(255)	NULLABLE	URL gambar utama artikel
status	ENUM	DEFAULT 'draft'	Status: draft, review, scheduled, published, archived
is_featured	BOOLEAN	DEFAULT false	Penanda artikel unggulan
view_count	BIGINT UNSIGNED	DEFAULT 0	Jumlah kunjungan artikel
published_at	TIMESTAMP	NULLABLE	Waktu publikasi artikel
created_at	TIMESTAMP	AUTO	Waktu pembuatan artikel
updated_at	TIMESTAMP	AUTO	Waktu update terakhir

C. Tabel categories

Tabel **categories** menyimpan kategori berita dengan dukungan hierarki parent-child menggunakan self-referencing.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik kategori
parent_id	UUID	FK → categories, NULL ON DELETE, NULLABLE	ID kategori induk (untuk sub-kategori)
name	VARCHAR(255)	NOT NULL	Nama kategori (Politik, Ekonomi, dll.)
slug	VARCHAR(255)	UNIQUE, NOT NULL	URL-friendly identifier

D. Tabel tags

Tabel `tags` menyimpan label-topik yang dapat dikaitkan dengan artikel.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik tag
name	VARCHAR(255)	UNIQUE, NOT NULL	Nama tag
slug	VARCHAR(255)	UNIQUE, NOT NULL	URL-friendly identifier

E. Tabel article_tags

Tabel junction untuk relasi many-to-many antara artikel dan tag.

Kolom	Tipe Data	Constraint	Deskripsi
article_id	UUID	PK, FK → articles, CASCADE ON DELETE	ID artikel
tag_id	UUID	PK, FK → tags, CASCADE ON DELETE	ID tag

F. Tabel comments

Tabel `comments` menyimpan komentar pembaca dengan dukungan nested reply menggunakan self-referencing `parent_id`.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik komentar
article_id	UUID	FK → articles, CASCADE ON DELETE	ID artikel yang dikomentari

user_id	BIGINT	FK → users, CASCADE ON DELETE	ID pengguna yang berkomentar
parent_id	UUID	FK → comments, NULL ON DELETE, NULLABLE	ID komentar induk (untuk reply)
content	TEXT	NOT NULL	Isi komentar
status	ENUM	DEFAULT 'pending'	Status: pending, approved, rejected
created_at	TIMESTAMP	AUTO	Waktu komentar dibuat

G. Tabel bookmarks

Tabel bookmarks menyimpan artikel favorit pembaca. Constraint unique pada (user_id, article_id) mencegah duplikasi.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik bookmark
user_id	BIGINT	FK → users, CASCADE ON DELETE	ID pembaca yang menyimpan
article_id	UUID	FK → articles, CASCADE ON DELETE	ID artikel yang disimpan
created_at	TIMESTAMP	AUTO	Waktu bookmark dibuat

H. Tabel article_revisions

Tabel article_revisions menyimpan riwayat perubahan artikel sebagai audit trail.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik revisi
article_id	UUID	FK → articles, CASCADE ON DELETE	ID artikel yang direvisi
edited_by	BIGINT	FK → users, RESTRICT ON DELETE	ID pengguna yang merevisi
title_snapshot	VARCHAR(255)	NOT NULL	Snapshot judul sebelum perubahan

content_snapshot	LONGTEXT	NOT NULL	Snapshot konten sebelum perubahan
change_note	VARCHAR(255)	NULLABLE	Catatan perubahan dari editor
created_at	TIMESTAMP	AUTO	Waktu revisi dibuat

I. Tabel `scheduled_articles`

Tabel `scheduled_articles` menyimpan jadwal publikasi artikel. Kolom `article_id` bersifat unique untuk memastikan satu artikel hanya memiliki satu jadwal aktif.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik jadwal
article_id	UUID	UNIQUE, FK → articles, CASCADE ON DELETE	ID artikel yang dijadwalkan
scheduled_by	BIGINT	FK → users, RESTRICT ON DELETE	ID user yang menjadwalkan
scheduled_at	TIMESTAMP	NOT NULL	Waktu yang dijadwalkan untuk publish
is_published	BOOLEAN	DEFAULT false	Flag apakah sudah diproses cron
created_at	TIMESTAMP	AUTO	Waktu penjadwalan dibuat

J. Tabel `search_indexes`

Tabel `search_indexes` menyimpan index pencarian full-text. Satu artikel memiliki tepat satu index.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik index
article_id	UUID	UNIQUE, FK → articles, CASCADE ON DELETE	ID artikel yang di-index
search_vector	TEXT	FULLTEXT INDEX	Gabungan judul + konten untuk pencarian
tags_cache	TEXT	NULLABLE	Nama tag dalam satu string

updated_at	TIMESTAMP	AUTO UPDATE	Waktu index terakhir diperbarui
------------	-----------	-------------	---------------------------------

K. Tabel media

Tabel **media** menyimpan referensi file media yang diupload.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik media
article_id	UUID	FK → articles, CASCADE ON DELETE, NULLABLE	ID artikel terkait (null jika library)
uploaded_by	BIGINT	FK → users, NULL ON DELETE, NULLABLE	ID user yang mengupload
file_url	VARCHAR(255)	NOT NULL	URL file media
media_type	ENUM	NOT NULL	Tipe: image, video, audio
alt_text	VARCHAR(255)	NULLABLE	Teks alternatif untuk aksesibilitas
is_library	BOOLEAN	DEFAULT false	Penanda media library (tanpa artikel)
created_at	TIMESTAMP	AUTO	Waktu upload

L. Tabel refresh_tokens

Tabel **refresh_tokens** menyimpan token refresh untuk autentikasi JWT. Token disimpan dalam bentuk hash SHA-256.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik token
user_id	BIGINT	FK → users, CASCADE ON DELETE	ID pemilik token
token_hash	VARCHAR(255)	NOT NULL	Hash SHA-256 dari refresh token
device_info	VARCHAR(255)	NULLABLE	Info perangkat (user agent)
ip_address	VARCHAR(45)	NULLABLE	Alamat IP perangkat

is_revoked	BOOLEAN	DEFAULT false	Status token revoked/aktif
expires_at	TIMESTAMP	NOT NULL	Waktu kedaluwarsa token
created_at	TIMESTAMP	AUTO	Waktu token dibuat

M. Tabel password_reset_tokens

Tabel `password_reset_tokens` menyimpan token untuk reset password. Menggunakan email sebagai primary key.

Kolom	Tipe Data	Constraint	Deskripsi
email	VARCHAR(255)	PK	Email pemilik token
token	VARCHAR(255)	NOT NULL	Hash token reset password
created_at	TIMESTAMP	NULLABLE	Waktu token dibuat

N. Tabel page_views

Tabel `page_views` mencatat statistik kunjungan halaman untuk analitik. Menggunakan auto-increment karena volume data tinggi.

Kolom	Tipe Data	Constraint	Deskripsi
id	BIGINT	PK, Auto Increment	Identifier unik page view
path	VARCHAR(255)	INDEX, NOT NULL	Path URL yang dikunjungi
ip_address	VARCHAR(45)	NULLABLE	Alamat IP pengunjung
user_agent	TEXT	NULLABLE	User agent browser
created_at	TIMESTAMP	AUTO	Waktu kunjungan
updated_at	TIMESTAMP	AUTO	Waktu update record

O. Tabel comment_rate_limits

Tabel `comment_rate_limits` membatasi jumlah komentar per user untuk mencegah spam.

Kolom	Tipe Data	Constraint	Deskripsi
id	UUID	PK	Identifier unik record

user_id	BIGINT	UNIQUE, FK → users, CASCADE ON DELETE	ID user yang dibatasi
comment_count	INT UNSIGNED	DEFAULT 0	Jumlah komentar dalam window
window_start	TIMESTAMP	AUTO	Waktu mulai window rate limit
is_blocked	BOOLEAN	DEFAULT false	Status blokir komentar
blocked_until	TIMESTAMP	NULLABLE	Waktu berakhirnya blokir

III.2.14 GitHub Repository dan Kontribusi

Seluruh kode sumber proyek dikelola menggunakan **Git** sebagai version control system dan di-hosting pada platform **GitHub**. Repository proyek berisi dua sub-direktori utama: **backend/** yang berisi kode sumber Laravel 12, dan **frontend/** yang berisi kode sumber Next.js 15. Pendekatan monorepo ini dipilih untuk memudahkan pengelolaan dependency antar kedua aplikasi dan memastikan sinkronisasi API contract antara frontend dan backend.

Berikut adalah tampilan halaman repository proyek di GitHub:

[Screenshot: Halaman utama repository GitHub yang menampilkan struktur folder proyek (backend/, frontend/, diagrams/, dll.) beserta README.md]

Gambar 3.5 Tampilan Repository Proyek di GitHub

Selama periode magang (9 Februari – 19 Juni 2026), penulis aktif melakukan commit ke repository. Total kontribusi penulis mencakup pengembangan fitur backend (scheduled publish, user management CRUD), frontend CMS dan portal berita, serta perancangan database migration. Berikut adalah grafik kontribusi commit penulis:

[Screenshot: GitHub Insights / Contributions graph yang menampilkan total commit, additions, dan deletions selama periode magang]

Gambar 3.6 Statistik Kontribusi Commit Penulis di GitHub

Penulis juga menerapkan workflow kolaborasi yang profesional menggunakan fitur **Pull Request** dan **Branch Protection**. Setiap fitur dikerjakan di branch terpisah (misalnya **feature/scheduled-publish** atau **feature/user-management**) dan di-merge ke branch **main** melalui pull request yang telah di-review oleh mentor. Berikut adalah tampilan daftar pull request yang telah di-merge:

[Screenshot: Daftar pull request yang telah di-merge di GitHub, menampilkan judul PR, branch asal, dan tanggal merge]

Gambar 3.7 Daftar Pull Request yang Telah Di-merge

Repository juga dilengkapi dengan file `.gitignore` untuk mencegah file-file sensitif atau tidak relevan (seperti `node_modules/`, `vendor/`, `.env`) ter-commit ke repository. Selain itu, file `composer.lock` dan `package-lock.json` tetap di-commit untuk memastikan konsistensi dependency antar lingkungan pengembangan. Berikut adalah rekapitulasi statistik kontribusi penulis selama periode magang:

Metric	Jumlah
Total Commits	[Isi dengan jumlah commit total]
Files Changed	[Isi dengan jumlah file yang diubah]
Additions (+)	[Isi dengan total baris kode yang ditambahkan]
Deletions (-)	[Isi dengan total baris kode yang dihapus]
Pull Requests Merged	[Isi dengan jumlah PR yang di-merge]

Tabel 2. Rekapitulasi Statistik Kontribusi Penulis di GitHub

[Screenshot: Halaman commit history yang menampilkan daftar commit penulis beserta pesan commit dan tanggal]

Gambar 3.8 Riwayat Commit Penulis di Repository

III.3 Pencapaian Hasil

Berdasarkan pelaksanaan proyek yang telah dilakukan, berikut adalah pencapaian yang telah diraih:

- 1. Struktur Proyek Berhasil Disusun** — Penulis berhasil menyusun seluruh struktur proyek dari awal, meliputi konfigurasi repository Git, pemilihan dan setup tech stack (Laravel 12, Next.js 15, Tailwind CSS), perancangan database komprehensif (CDM, PDM, ERD) yang terdiri dari 15 tabel, serta dokumentasi perancangan sistem menggunakan diagram UML.
- 2. Fitur Cronjob Scheduled Publish Berfungsi** — Fitur penjadwalan publikasi artikel berhasil diimplementasikan dan berjalan dengan baik. Cron job berhasil memproses artikel terjadwal secara otomatis setiap menit dan melakukan reindex search index.
- 3. Fitur Manajemen User (CRUD) Berfungsi** — Seluruh operasi CRUD untuk manajemen user oleh admin berhasil diimplementasikan: pembuatan user baru dengan auto-generate password dan pengiriman email kredensial, pengeditan data user dengan mekanisme revoke token otomatis saat perubahan role, serta penghapusan user dengan penanganan dampak.
- 4. Integrasi dengan Sistem Keseluruhan** — Kedua fitur telah terintegrasi dengan baik dalam ekosistem CMS. Fitur Scheduled Publish terintegrasi dengan editorial

workflow, dan fitur Manajemen User terintegrasi dengan sistem autentikasi JWT serta kontrol akses berbasis peran.

Dalam proses pelaksanaan, penulis juga memperoleh beberapa pengalaman berharga, antara lain pemahaman mendalam mengenai pola arsitektur Service-Repository, pengalaman merancang database dengan UUID sebagai primary key, pemahaman mengenai mekanisme cron job dan database transaction, serta pengalaman menangani edge case pada operasi CRUD.

III.4 Hasil Testing

Pada bagian ini disajikan hasil pengujian terhadap fitur-fitur yang telah diimplementasikan. Pengujian dilakukan untuk memastikan seluruh fitur berjalan sesuai spesifikasi yang telah ditetapkan. Berikut adalah tabel pengujian beserta dokumentasi screenshot hasil pengujian.

III.4.1 Testing Cronjob Scheduled Publish

No	Skenario Pengujian	Langkah	Hasil yang Diharapkan	Screenshot
1	Jadwalkan artikel 5 menit ke depan	Buat artikel, set status = scheduled, atur waktu 5 menit ke depan	Artikel tersimpan dengan status scheduled	
2	Tunggu cron job berjalan	Jalankan php artisan <code>articles:publish-scheduled</code> setelah waktu tiba	Command memproses artikel terjadwal	
3	Verifikasi status artikel berubah	Cek status artikel di database setelah cron berjalan	Status berubah menjadi published, published_at terisi	
4	Verifikasi search index ter-update	Cek tabel <code>search_indexes</code> untuk artikel yang baru published	Artikel muncul di <code>search_indexes</code> dengan <code>search_vector</code> yang benar	
5	Jalankan cron saat tidak ada artikel	Jalankan command ketika tidak ada artikel terjadwal	Output: Tidak ada artikel yang perlu dipublish	
6	Artikel dengan waktu > 5 menit	Jadwalkan artikel 10 menit ke depan, jalankan cron langsung	Artikel belum dipublish (belum waktunya)	

III.4.2 Testing Manajemen User — Create

No	Skenario Pengujian	Langkah	Hasil yang Diharapkan	Screenshot
1	Buat user baru dengan data valid	Isi form: nama, email unik, role jurnalis, klik simpan	User berhasil dibuat, email kredensial terkirim	
2	Buat user dengan email duplikat	Isi form dengan email yang sudah terdaftar	Tampil error: Email sudah terdaftar	
3	Buat user tanpa isi nama	Kosongkan field nama, klik simpan	Tampil validasi: field nama wajib diisi	
4	Buat user dengan role editor	Isi form lengkap, pilih role editor	User baru memiliki role editor	
5	Verifikasi email terkirim	Cek inbox email user baru	Email berisi nama, email, password sementara, dan role	

III.4.3 Testing Manajemen User — Update

No	Skenario Pengujian	Langkah	Hasil yang Diharapkan	Screenshot
1	Edit nama user	Ubah nama user, klik simpan	Nama berhasil diperbarui	
2	Edit role user	Ubah role dari journalist ke editor	Role berubah, token user di-revoke (logout paksa)	
3	Nonaktifkan user	Set is_active = false	User dinonaktifkan, tidak bisa login	
4	Edit email ke email yang sudah ada	Ubah email ke email user lain	Tampil error: Email sudah terdaftar	
5	Coba edit akun sendiri	Admin mencoba edit akunnya sendiri	Tampil error: Tidak dapat mengubah akun sendiri	

III.4.4 Testing Manajemen User — Delete

No	Skenario Pengujian	Langkah	Hasil yang Diharapkan	Screenshot
1	Hapus user biasa	Pilih user, klik hapus, konfirmasi	User terhapus, artikel diarsipkan	
2	Coba hapus akun sendiri	Admin mencoba hapus akunnya sendiri	Tampil error: Tidak dapat menghapus akun sendiri	

3	Verifikasi artikel terarsipkan	Cek artikel milik user yang dihapus	Semua artikel berstatus archived	
4	Verifikasi token di-revoke	Cek refresh_tokens user yang dihapus	Semua token sudah revoked	

III.4.5 Testing Manajemen User — Read

No	Skenario Pengujian	Langkah	Hasil yang Diharapkan	Screenshot
1	Lihat daftar user	Buka halaman Pengguna sebagai admin	Tampil daftar seluruh user dengan nama, email, role, status	
2	Lihat detail user	Klik salah satu user	Tampil detail informasi user	
3	Akses tanpa role admin	Login sebagai journalist, akses /api/users	Akses ditolak (403 Forbidden)	

III.4.6 Testing Pengiriman Email Kredensial

No	Skenario Pengujian	Langkah	Hasil yang Diharapkan	Screenshot
1	Email terkirim saat buat user baru	Admin membuat user baru dengan email valid	Email masuk ke inbox user baru berisi nama, email, password sementara, dan role	
2	Isi email kredensial benar	Buka email yang diterima user baru	Password sementara sesuai yang digenerate sistem, role sesuai yang dipilih admin	
3	Login dengan password sementara	Gunakan email dan password sementara dari email untuk login	Login berhasil, user masuk ke dashboard sesuai role	
4	Ganti password setelah login pertama	Login dengan password sementara, buka menu Profil, ubah password	Password berhasil diubah, password lama tidak bisa digunakan lagi	

III.4.7 Testing Database Transaction dan Konsistensi Data

Pengujian database transaction dilakukan untuk memastikan bahwa operasi yang melibatkan beberapa tabel berjalan secara atomik dan konsisten. Pengujian ini berfokus pada fitur Scheduled Publish yang menggunakan `DB::transaction()`.

No	Skenario Pengujian	Langkah	Hasil yang Diharapkan	Screenshot
1	Publish artikel — seluruh operasi berhasil	Jadwalkan artikel, tunggu cron berjalan, cek ketiga tabel	Status updated, is_published=true, search index terisi	
2	Simulasi kegagalan reindex	Nonaktifkan tabel search_indexes sementara, jalankan cron	Transaction rollback, status artikel tetap scheduled	
3	Publish multiple articles sekaligus	Jadwalkan 3 artikel pada waktu yang sama, jalankan cron	Ketiga artikel berhasil published secara bersamaan	
4	Idempotensi — cron berjalan dua kali	Jalankan cron setelah artikel sudah published	Tidak ada perubahan data, output: 0 artikel	
5	Konsistensi published_at timestamp	Cek kolom published_at setelah auto-publish	Timestamp sesuai waktu eksekusi cron, bukan waktu penjadwalan	

III.4.8 Testing API Endpoint dengan Postman

Selain pengujian melalui antarmuka frontend, penulis juga melakukan pengujian langsung terhadap API endpoint menggunakan **Postman** untuk memverifikasi request dan response secara detail. Pengujian ini mencakup validasi status code, struktur response JSON, dan penanganan error.

No	Endpoint	Skenario	Expected Response	Screenshot
1	POST /api/users	Kirim request dengan data valid	201 Created, body berisi id, name, email, role	
2	POST /api/users	Kirim email yang sudah terdaftar	409 Conflict, error = EMAIL_ALREADY_EXISTS	
3	PATCH /api/users/{id}	Ubah role user	200 OK, role berubah, token di-revoke	
4	DELETE /api/users/{id}	Hapus user yang memiliki artikel	200 OK, artikel user berstatus archived	
5	GET /api/users	Akses tanpa token JWT	401 Unauthorized	

6	GET /api/users	Akses dengan role journalist	403 Forbidden	
7	DELETE /api/users/{self}	Admin menghapus akun sendiri	403 CANNOT_MODIFY_SELF	
8	PATCH /api/cms/articles/{id}/status	Set status=scheduled dengan waktu valid	200 OK, status berubah ke scheduled	

Pengujian menggunakan Postman memberikan visibilitas penuh terhadap request dan response HTTP, memungkinkan penulis memverifikasi bahwa setiap endpoint mengembalikan status code yang tepat, struktur JSON yang konsisten, dan pesan error yang informatif. Postman Collection yang berisi seluruh test case juga disimpan di repository untuk memudahkan pengujian regresi di masa depan.

III.5 Rekognisi Mata Kuliah

Kegiatan Program Pembelajaran di Luar Kampus (PLK) yang dilaksanakan di PT. Ketik Media Siber telah memberikan pengalaman praktis yang relevan dengan beberapa mata kuliah yang telah atau sedang ditempuh oleh penulis di Program Studi Informatika. Berikut adalah penjabaran keterkaitan antara pekerjaan yang dilakukan selama magang dengan masing-masing mata kuliah yang direkognisi.

1. Analisis Kebutuhan

Mata kuliah **Analisis Kebutuhan** membahas teknik-teknik identifikasi, analisis, dan dokumentasi kebutuhan sistem perangkat lunak. Selama kegiatan magang, penulis menerapkan kompetensi dari mata kuliah ini secara langsung pada tahap awal pengembangan Sistem Portal Berita dan CMS Redaksi.

Penulis melakukan analisis kebutuhan sistem dengan mempelajari alur kerja redaksi PT. Ketik Media Siber secara menyeluruh, mulai dari proses penulisan berita oleh jurnalis, proses peninjauan dan persetujuan oleh editor, hingga proses publikasi artikel. Berdasarkan analisis tersebut, penulis mengidentifikasi dua kebutuhan fungsional utama yang menjadi tanggung jawab pengembangan, yaitu fitur **penjadwalan publikasi artikel otomatis (scheduled publish)** yang memungkinkan editor mengatur waktu tayang artikel dan sistem akan mempublikasikannya secara otomatis menggunakan cron job, serta fitur **manajemen pengguna (user management)** yang memungkinkan admin mengelola akun pengguna meliputi pembuatan, pengeditan, penonaktifan, dan penghapusan akun dengan kontrol akses berbasis role.

Selain itu, penulis juga melakukan analisis kebutuhan non-fungsional, seperti kebutuhan akan arsitektur sistem yang modular menggunakan pendekatan *decoupled architecture* dan *Service-Repository Pattern*, kebutuhan akan perancangan database yang komprehensif menggunakan CDM, PDM, dan ERD dengan 15 tabel, serta kebutuhan keamanan melalui mekanisme autentikasi JWT dan role-based access control. Hasil analisis kebutuhan kemudian didokumentasikan dalam bentuk diagram UML, meliputi *Sequence Diagram* untuk menggambarkan interaksi antar komponen sistem, *Activity Diagram* untuk menggambarkan alur proses secara sistematis, dan *Class Diagram* untuk menggambarkan struktur data dan relasi antar entitas.

2. Pemrograman API

Mata kuliah **Pemrograman API** membahas perancangan dan pengembangan Application Programming Interface (API) yang mengikuti standar RESTful, termasuk desain endpoint, penanganan request dan response, validasi data, serta mekanisme autentikasi. Kompetensi dari mata kuliah ini diterapkan secara intensif oleh penulis pada tahap pengembangan backend sistem.

Penulis membangun beberapa endpoint REST API menggunakan framework **Laravel 12** dengan menerapkan *Service-Repository Pattern* untuk memisahkan logika bisnis dari akses data. Untuk fitur *scheduled publish*, penulis membangun endpoint `PATCH /api/cms/articles/{id}/status` yang memungkinkan editor mengubah status artikel menjadi *scheduled* dengan menyertakan parameter `scheduled_at`. Penulis juga membangun *Artisan Command* (`articles:publish-scheduled`) yang dipicu oleh Laravel Task Scheduler setiap menit untuk memproses artikel yang sudah jatuh tempo, serta `ScheduledPublishService` yang berisi logika bisnis untuk mempublikasikan artikel secara otomatis dalam satu *database transaction*.

Untuk fitur manajemen user, penulis membangun operasi CRUD lengkap yang hanya dapat diakses oleh admin. Endpoint yang dikembangkan meliputi `POST /api/users` untuk membuat akun baru dengan *auto-generate password* 12 karakter dan pengiriman kredensial melalui email menggunakan `NewUserCredentials Mailable`, `PUT /api/users/{id}` untuk memperbarui data pengguna dengan mekanisme *revoke token* otomatis apabila terjadi perubahan role, `PATCH /api/users/{id}/deactivate` untuk menonaktifkan akun, serta `DELETE /api/users/{id}` untuk menghapus akun beserta pengarsipan seluruh artikel milik pengguna tersebut. Penulis juga merancang template email menggunakan *Blade template* (`emails.new-user-credentials`) yang berisi informasi kredensial akun (email, password sementara, dan role) serta peringatan agar user segera mengganti password setelah login pertama kali. Seluruh endpoint dilengkapi dengan middleware autentikasi JWT dan validasi role untuk memastikan keamanan akses.

3. Uji Coba dan Implementasi

Mata kuliah **Uji Coba dan Implementasi** membahas metodologi pengujian perangkat lunak, teknik implementasi sistem, serta strategi deployment. Kompetensi dari mata kuliah ini diterapkan oleh penulis pada tahap implementasi dan pengujian fitur-fitur sistem yang telah dikembangkan.

Pada tahap implementasi, penulis mengimplementasikan seluruh rancangan sistem ke dalam kode program menggunakan **Laravel 12** untuk backend dan **Next.js 15** untuk frontend CMS. Implementasi fitur **scheduled publish** mencakup pembangunan Artisan Command sebagai *entry point* cron job, `ScheduledPublishService` untuk menangani logika bisnis publikasi otomatis, serta konfigurasi Laravel Task Scheduler. Implementasi fitur manajemen user mencakup pembangunan `UserController`, `UserService`, dan `UserRepository` yang menangani seluruh operasi CRUD dengan penanganan edge case seperti validasi duplikasi email, pencegahan admin menghapus akun sendiri, serta pengarsipan artikel milik user yang dihapus.

Pada tahap uji coba, penulis melakukan pengujian fungsional terhadap seluruh fitur yang telah diimplementasikan. Pengujian fitur **scheduled publish** mencakup pengujian penjadwalan artikel dengan waktu lebih dari 5 menit ke depan, verifikasi perubahan status artikel secara otomatis oleh cron job ketika jadwal tiba, pengujian validasi waktu minimal penjadwalan, pengujian pembatalan jadwal, serta verifikasi pembaruan search index. Pengujian fitur **manajemen user** mencakup pengujian pembuatan user baru, validasi duplikasi email, verifikasi pengiriman email kredensial, pengujian pengeditan data termasuk perubahan role dan penonaktifan akun, pengujian penghapusan user beserta penanganan dampaknya terhadap data terkait, serta pengujian kontrol akses untuk memastikan hanya admin yang dapat mengakses fitur ini. Penulis juga melakukan pengujian khusus terhadap fitur **pengiriman email**, meliputi verifikasi email terkirim saat pembuatan user baru, validasi isi email kredensial (nama, email, password sementara, dan role), pengujian login menggunakan password sementara yang diterima melalui email, serta pengujian perubahan password setelah login pertama kali. Hasil pengujian menunjukkan bahwa seluruh fitur berfungsi sesuai spesifikasi yang telah ditetapkan pada tahap analisis kebutuhan.

BAB IV

PENUTUP

IV.1 Kesimpulan

Berdasarkan pelaksanaan Program Pembelajaran di Luar Kampus (PLK) yang telah dilakukan di PT. Ketik Media Siber, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Kegiatan magang memberikan pengalaman nyata dalam mengembangkan sistem portal berita dan CMS redaksi secara profesional, mulai dari tahap perancangan hingga implementasi fitur.
2. Penerapan arsitektur decoupled dengan Service-Repository Pattern terbukti efektif dalam membangun sistem yang modular, mudah dipelihara, dan dapat dikembangkan lebih lanjut.
3. Fitur cronjob scheduled publish berhasil diimplementasikan dengan baik menggunakan Artisan Command dan database transaction, memastikan artikel terpublikasi secara otomatis dan konsisten.
4. Fitur manajemen user (CRUD) berhasil menyediakan mekanisme pengelolaan pengguna yang aman, dilengkapi dengan auto-generate password, pengiriman email kredensial, dan penanganan dampak penghapusan user.
5. Perancangan database yang komprehensif menggunakan CDM, PDM, dan ERD menjadi fondasi penting yang menjamin integritas dan efisiensi data dalam sistem.

IV.2 Saran

Beberapa saran yang dapat dipertimbangkan untuk pengembangan selanjutnya adalah:

1. Penambahan fitur audit log untuk melacak seluruh perubahan yang dilakukan oleh admin pada sistem.
2. Implementasi soft delete pada tabel users untuk mempertahankan riwayat pengguna meskipun telah dihapus.
3. Penambahan fitur bulk action pada manajemen user untuk efisiensi pengelolaan pengguna dalam jumlah besar.
4. Pengembangan fitur notifikasi real-time untuk memberi tahu jurnalis ketika artikelnya dipublikasikan secara terjadwal.
5. Migrasi database dari SQLite ke PostgreSQL untuk mendukung skala produksi yang lebih besar.

Referensi

- [1] Laravel Documentation, “Laravel 12 — The PHP Framework for Web Artisans,” laravel.com, 2025.
- [2] Next.js Documentation, “Next.js by Vercel — The React Framework,” nextjs.org, 2025.
- [3] Tailwind CSS, “Tailwind CSS — Rapidly build modern websites,” tailwindcss.com, 2025.
- [4] PHP Documentation, “PHP Manual,” php.net, 2025.
- [5] TypeScript Documentation, “TypeScript: JavaScript With Syntax For Types,” typescriptlang.org, 2025.

Lampiran A. Log Activity

Berikut adalah log activity kegiatan selama mengikuti Program Magang Mandiri di PT. Ketik Media Siber:

Minggu/Tanggal	Kegiatan	Hasil	Validasi Dosen Pembimbing
Minggu Ke-1 / 9-14 Feb 2026	Mengikuti hari pertama magang dengan sesi pengenalan bersama tim dan pembimbing lapangan, serta mendapatkan penjelasan mengenai sistem kerja dan agenda kegiatan selama magang. Mempelajari dasar-dasar penulisan berita, teknik foto jurnalistik, dan melakukan praktik liputan sederhana.	Memahami alur kerja dan pembagian tugas selama magang, serta memperoleh pengetahuan dasar mengenai penulisan berita dan teknik pengambilan foto jurnalistik.	
Minggu Ke-2 / 16-21 Feb 2026	Melakukan revisi berita berdasarkan arahan pembimbing, mempelajari tata cara upload artikel melalui CMS website, serta mulai menyusun berita profil dan berita harian bertema Ramadan.	Tulisan berita menjadi lebih rapi dan sesuai standar penulisan. Memahami proses publikasi berita melalui CMS serta mampu membuat dan mengunggah berita secara mandiri.	

Minggu Ke-3 / 23-28 Feb 2026	Melanjutkan penulisan dan penyempurnaan berita profil serta rutin membuat berita harian Ramadan sesuai target yang diberikan. Mengikuti pemaparan materi dari perwakilan DPRD Kota Malang.	Berita harian Ramadan berhasil diselesaikan sesuai target. Mendapat pengalaman lebih dalam mengenai proses produksi berita.	
Minggu Ke-4 / 2-7 Mar 2026	Mendapatkan pengarahan dari pembimbing IT mengenai proyek yang akan dikerjakan. Penjelasan mengenai penggunaan Git dan GitHub dalam dunia kerja.	Memahami dasar penggunaan Git dan GitHub serta mengetahui alur kerja kolaborasi proyek.	
Minggu Ke-5 / 9-14 Mar 2026	Mulai mengerjakan project website dengan melakukan penambahan data, mempelajari struktur halaman website, serta pengelolaan file project menggunakan GitHub.	Memahami proses input dan pengelolaan data pada website serta mulai terbiasa menggunakan GitHub.	
Minggu Ke-6 / 16-21 Mar 2026	Melanjutkan pengerjaan project website dengan pembaruan data, perbaikan tampilan, serta mempelajari workflow tim seperti pull request dan merge repository.	Menambah pemahaman mengenai workflow GitHub serta mampu melakukan pengelolaan data website dengan lebih terstruktur.	
Minggu Ke-7 / 23-28 Mar 2026	Membantu pengecekan dan revisi data yang telah diinput, serta memperbaiki kesalahan tampilan dan penulisan pada website.	Data website menjadi lebih lengkap dan rapi. Kemampuan revisi data dan perbaikan tampilan semakin meningkat.	

Minggu Ke-8 / 30 Mar-4 Apr 2026	Melanjutkan pengembangan dan pengelolaan website dengan menambahkan data baru, update konten, serta pengecekan hasil akhir project.	Memahami proses pengelolaan dan pengembangan website secara lebih detail.	
Minggu Ke-9 / 6-11 Apr 2026	Mulai merancang dan membuat UI/UX untuk website Klojen dengan membuat desain tampilan halaman beranda, daftar lokasi, detail tempat, serta navigasi website.	Berhasil membuat rancangan dan tampilan awal UI/UX website Klojen yang responsif dan user friendly.	
Minggu Ke-10 / 13-18 Apr 2026	Melakukan perancangan sistem menggunakan diagram UML (Use Case, Activity, Sequence, Class Diagram) serta perancangan database (ERD, CDM, PDM).	Berhasil menyusun dokumentasi perancangan sistem berupa diagram UML dan desain database.	
Minggu Ke-11 / 20-25 Apr 2026	Setup project website Klojen dengan menyiapkan tech stack (Laravel 12, Next.js 15, Tailwind CSS), konfigurasi struktur project, dan pembuatan layout dasar.	Project berhasil di-setup. Struktur awal project dan layout dasar website telah berhasil dibuat.	
Minggu Ke-12 / 27 Apr-2 Mei 2026	Pengembangan tampilan front-end berdasarkan desain UI/UX yang telah dibuat. Membuat halaman beranda, kategori, jelajah, dan detail.	Halaman beranda, kategori, jelajah, dan detail berhasil dikembangkan dengan tampilan yang terstruktur dan modern.	
Minggu Ke-13 / 4-9 Mei 2026	Melanjutkan pengembangan front-	Fitur review, galeri, dan interaktivitas	

	end dengan membuat fitur review, galeri foto, dan interaktivitas pada halaman detail.	berhasil dikembangkan.	
Minggu Ke-14 / 11-16 Mei 2026	Menyempurnaan halaman kategori, trending, serta perbaikan tampilan responsif berdasarkan evaluasi pembimbing.	Halaman kategori dan trending berhasil disempurnakan dengan tampilan yang responsif.	
Minggu Ke-15 / 18-23 Mei 2026	Perancangan dan pengembangan fitur baru menggunakan Figma. Mulai integrasi data dari API untuk menampilkan konten secara dinamis.	Desain fitur berhasil dibuat dan halaman website berhasil terhubung dengan API.	
Minggu Ke-16 / 25-30 Mei 2026	Pengembangan fitur pencarian, filter data, serta pengujian integrasi API dan perbaikan tampilan.	Fitur pencarian dan filter berhasil dikembangkan. Integrasi data berjalan dengan baik.	
Minggu Ke-17 / 1-6 Jun 2026	Penyempurnaan fitur website: memperbaiki bug, optimasi komponen, serta memperbaiki struktur kode.	Beberapa bug berhasil diperbaiki dan performa website menjadi lebih baik.	
Minggu Ke-18 / 8-13 Jun 2026	Testing fitur website, pengecekan responsivitas, serta penyesuaian data dan endpoint API.	Fitur website berhasil diuji dan berjalan dengan baik. Sistem menjadi lebih stabil.	
Minggu Ke-19 / 15-19 Jun 2026	Finalisasi project: pengecekan keseluruhan fitur, debugging, penyempurnaan tampilan, penyusunan	Project berhasil diselesaikan. Dokumentasi dan bahan presentasi berhasil disusun.	

	dokumentasi, dan persiapan presentasi.		
--	---	--	--